

DeGro: Grounded Target-Determinacy Verification for Executable Mathematical Reasoning

Anonymous Author(s)
Submission Id: ANONYMOUS

Abstract

Executable mathematical reasoning can eliminate arithmetic and runtime errors while still silently solving the wrong formalization. We introduce *DeGro*, a grounded determinacy verification-and-repair method that asks whether every feasible model of a structured specification agrees on the requested target. A satisfiable two-copy query establishes target ambiguity; DeGro then attempts a source-grounded repair or abstains. We evaluate DeGro on 120 preregistered omission/underspecification pairs, with the initial formalization held fixed within each pair. On GPT-OSS 20B, DeGro improves final decision accuracy over grounded self-review by 5.0 percentage points (95% paired-bootstrap CI [0.4, 9.6]; exact McNemar $p = .050$, Holm-adjusted $p = .100$). The corresponding Gemma 4 31B effect is -0.4 points (95% CI $[-3.3, 2.5]$; $p = 1.0$), so the confirmatory evidence is model-dependent. A cumulative 300-pair analysis yields effects of $+5.0$ and $+1.7$ points, respectively. DeGro certifies determinacy under a representation, not semantic equivalence between that representation and natural language.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Intelligent agents**; *Natural language processing*.

Keywords

neuro-symbolic reasoning, mathematical agents, verification, counterexamples, abstention

ACM Reference Format:

Anonymous Author(s). 2027. DeGro: Grounded Target-Determinacy Verification for Executable Mathematical Reasoning. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 10 pages.

1 Introduction

Language models increasingly delegate mathematical computation to executable programs and symbolic solvers. Program-aided reasoning can reduce arithmetic mistakes, and SymCode converts a problem into a self-contained SymPy program with execution-based repair [20, 37]. Yet execution establishes only that code runs. A feasible solver output does not establish that the program contains every condition needed to justify the answer.

Consider “Ten tasks are divided between A and B. A receives two more than B. How many does A receive?” If a generated formalization contains only $a + b = 10$, it executes and has feasible

solutions. However, $(a, b) = (4, 6)$ and $(6, 4)$ yield different values of the target a . The same underconstrained formalization is faithful for the shorter source “Ten tasks are divided between A and B. How many does A receive?”; here adding $a = b + 2$ would be fabrication. A trustworthy agent must distinguish a missing translation from missing information.

We study whether target-directed solver evidence enables this distinction. DeGro searches for two feasible assignments that disagree specifically on the requested target and invokes a repair-or-abstain policy grounded in the original source. The method is specialized: it detects underconstraint visible as target ambiguity and does not claim full semantic verification.

Our contributions are: (1) an operational formulation of target determinacy for executable LLM-generated mathematical specifications; (2) target-divergent evidence and a solver-validated grounded repair protocol; (3) a preregistered paired evaluation that controls the formalization while varying only source support; and (4) a frozen, blinded natural-error cohort and adjudication protocol that distinguish target-critical omissions from other formalization errors. The confirmatory results establish a benefit for one model but not the other, sharpening the claim from universal witness benefit to model-dependent value of grounded determinacy feedback.

Research questions. We ask: **RQ1**, whether determinacy information improves the joint repair-or-abstain decision; **RQ2**, whether grounding reduces unsupported additions; **RQ3**, whether concrete target-divergent models help beyond a bare non-uniqueness verdict; and **RQ4**, how often the present representation reaches a conclusive solver state on naturally written mathematics. RQ1–RQ3 are evaluated on the controlled benchmark. RQ4 is a coverage audit, not evidence of semantic faithfulness.

2 Related Work

Executable mathematical reasoning. Chain-of-thought elicits intermediate natural-language reasoning [48], while PAL and Program-of-Thoughts delegate computation to generated programs [9, 20]. Toolformer and ReAct more broadly integrate model reasoning with external tools [41, 53]; MathCoder and ToRA specialize this pattern to mathematical code and tool interaction [24, 47]. SymCode treats an executable symbolic program as the reasoning artifact and repairs runtime failures [37]. DeGro addresses a complementary silent failure: executable code whose feasible region does not determine the query.

Verifier-based selection ranks candidate solutions using learned outcome or process signals [13, 33, 45]. Self-verification, Self-Refine, and CRITIC instead ask a model to inspect or revise its output, sometimes with tool feedback [23, 35, 49]; controlled studies also show that intrinsic self-correction can fail without reliable external feedback [26]. These approaches can improve answer accuracy without resolving the translation boundary studied here: a runtime



This work is licensed under a Creative Commons Attribution International 4.0 License.

or learned verifier may accept a program that omits a necessary source condition.

Relational verification. Self-composition reduces a two-execution information-flow property to ordinary safety checking by composing a program with a renamed copy [8, 44]. Product programs and Cartesian Hoare logic make such relational proofs more modular and scalable [7, 43]. Hyperproperties and k -safety generalize the view from single traces to sets or tuples of traces [12]; HyperLTL and its model-checking algorithms provide a temporal specification route [11, 19]. Equation (1) uses the same two-copy pattern, but the relation of interest is agreement on a mathematical target across feasible assignments rather than noninterference across executions. This connection supplies established formal-methods terminology without implying a general relational verifier.

Selective prediction and abstention. Selective classification formalizes the risk–coverage tradeoff induced by a reject option [14, 18, 21]; SelectiveNet learns prediction and selection jointly [22], while learning-to-defer routes difficult cases to an expert [36]. Conformal risk control offers distribution-free calibration of selected actions under stated assumptions [3]. For language models, self-knowledge and semantic uncertainty provide confidence signals over answers or meaning-equivalent generations [29, 30]. DeGro instead derives abstention from a symbolic condition: the source does not license a constraint needed to determine the target. We therefore report coverage and conditional accuracy separately from end-to-end accuracy, and treat UNKNOWN and UNSUPPORTED as coverage failures rather than incorrect proofs.

Solver-guided diagnosis. ReLoop uses behavioral parameter perturbations to expose semantically incorrect optimization models that nevertheless execute [31]. SymDiag performs step-level satisfiability and entailment checks over translated reasoning traces and returns structured diagnostic evidence [15]. Minimal-core-guided repair supplies proof-like feedback for overconstrained, unsatisfiable formalizations [40]. DeGro instead considers the underconstrained regime: satisfiable specifications with target-divergent models. Its central decision is whether the source supports repair or requires abstention.

The mechanism is also related to counterexample-guided abstraction and synthesis loops, which alternate candidates with mechanically produced refutations [10, 42]. Its witness is not a synthesis counterexample, however: it establishes target ambiguity and leaves source-grounded diagnosis to the repair policy. DeGro supplies a typed solver state and subjects every proposed change to deterministic gates, changing the language model’s role from verifier to repair proposer.

Specification repair and semantic faithfulness. Classical diagnosis identifies minimal faulty assumptions, while abductive inference searches for hypotheses that explain or restore a desired property [17, 39]. Minimal unsatisfiable subsets and failure-inducing input reduction likewise seek small explanatory causes [32, 56]. SAT-based model repair modifies a transition system until it satisfies a formal property [4], and recent work studies LLM repair of declarative Alloy specifications [2]. Our acceptance gate is narrower: it adds only a cited, non-redundant, consistency-preserving constraint that restores target determinacy.

Autoformalization translates informal mathematics into formal statements [27, 50]. Benchmarks and corpora such as miniF2F, ProofNet, Lean Workbook, and FormalMATH expand the evaluation and training substrate [5, 54, 55, 57], while LeanDojo, Llemma, Draft-Sketch-Prove, and DeepSeek-Prover advance theorem-proving infrastructure and models [6, 28, 51, 52]. Faithful Chain-of-Thought makes intermediate computation executable [34], but syntactic validity and prover acceptance still do not establish that the formal object preserves the source meaning. DeGro checks one necessary relational consequence of a representation and leaves source-to-formal equivalence outside its guarantee.

Missing-information benchmarks. MIRA-Math contains 2,310 generated instances in 22 families, each hiding one necessary atomic fact from the solver-facing view while retaining it for an information holder [1]. Earlier algebra-word-problem work also argues for evaluating derivations rather than answers alone [46]. We derive paired cases from validated latent states, but evaluate representation repair and grounded abstention rather than claiming the benchmark as a contribution.

3 Problem Formulation

Let a natural-language problem be P . A model produces $F = (V, D, C, q)$, where V are variables, D their domains, C constraints, and $q(V)$ the requested expression. Define

$$\text{Sol}(F) = \{m \mid m \models D \wedge C\}.$$

The target is *determinate under F* iff

$$\forall m_1, m_2 \in \text{Sol}(F), \quad q(m_1) = q(m_2).$$

This is weaker than solution uniqueness: $x = 5, y \geq 0$ has many models, but target $q = x$ is determinate.

Construct renamed copies $F^{(1)}$ and $F^{(2)}$ and query

$$\Phi(F, q) = F^{(1)} \wedge F^{(2)} \wedge q(V^{(1)}) \neq q(V^{(2)}). \quad (1)$$

If (1) is satisfiable, its two projected models form a *target-divergent witness pair*. If it is unsatisfiable and F itself is satisfiable, the target is determinate.

Guarantee. For a sound solver and supported theory, an UNSAT result for Eq. (1), together with $\text{SAT}(F)$, implies that all models of F agree on q . This follows immediately because a disagreeing pair would satisfy Eq. (1). A timeout or UNKNOWN result proves nothing and is never mapped to determinacy.

Soundness boundary. The guarantee is relative to F . It does not establish $F \equiv P$. An unsupported extra constraint may make q determinate while mistranslating the source. Provenance gates reduce, but do not formally eliminate, this semantic gap.

3.1 Operational properties

Target determinacy is query-relative. A scheduling model can leave individual assignments free while determining total cost; requiring full model uniqueness would reject such a valid answer. A returned witness pair is independently checkable by substituting both assignments into $D \wedge C$ and comparing their target values. Moreover, determinacy is monotone under consistent strengthening: if every model of F agrees on q , then every model of $F \wedge c$ agrees on the same value. The current policy therefore triggers additions only

for ambiguous specifications. Repairing a determinate but incorrect representation may require deleting or revising an existing constraint and is outside the method.

The converse is unsafe: an arbitrary strengthening can force determinacy. Adding $a = 6$ to $a + b = 10$ determines a whether or not the source licenses that fact. We therefore separate *formal validity*—parseability, consistency, novelty, and sufficiency—from *evidential validity*, which requires source support.

3.2 Worked example

For $V = \{a, b\}$, nonnegative integer domains, $C = (a + b = 10)$, and $q = a$, the duplicated query is

$$a_1 + b_1 = 10 \wedge a_2 + b_2 = 10 \wedge a_1 \neq a_2.$$

A solver can return (4, 6) and (6, 4). If the source includes “A receives two more than B,” then $a = b + 2$ is source-supported, consistent, non-redundant, and sufficient; the repaired target is six. Without that sentence, the same equation would be a fabricated assumption, so the appropriate external action is abstention. Neither decision exposes the hidden gold answer to the repair model.

4 DeGro

4.1 Structured representation

The agent emits one typed MODEL SPEC containing variable sorts and bounds, constraint expressions with source spans, and a target expression. Deterministic compilers translate the same object to the computational and verification backends. This avoids divergence between independently generated SymPy and SMT programs. The initial supported fragment includes integer, real, and Boolean variables; arithmetic equalities and inequalities; finite domains; and selected modular constraints. Unsupported syntax yields NOT-SUPPORTED.

4.2 Closed-loop policy

After execution, DeGro first tests $\text{SAT}(F)$. Unsatisfiability yields INCONSISTENT; unknown yields UNKNOWN. Otherwise it evaluates Eq. (1). Unsatisfiability permits an answer under the current formalization. Satisfiability yields two assignments with different target values and triggers grounded diagnosis.

The repair model receives P , C , q , and the witness pair, and must return either ADD-CONSTRAINT with an exact supporting span and formal expression, or ABSTAIN. A candidate c is accepted only if: (i) it compiles; (ii) its cited span occurs in P , or it uses a declared domain-semantics rule; (iii) $F \wedge c$ is satisfiable; (iv) $F \not\models c$; and (v) rechecking restores determinacy or yields a new witness for the next bounded round. We use at most two repair rounds. The external statuses are ANSWER, UNDERSPECIFIED, INCONSISTENT, UNKNOWN, and UNSUPPORTED.

Exact substring provenance is deliberately conservative and easy to audit. It does not prove that a cited sentence was translated correctly; semantic validation of the proposed equation remains a failure mode.

4.3 Implementation and trusted boundary

The reference implementation compiles a restricted expression abstract syntax tree to Z3 [16]. It rejects attribute access, arbitrary

function calls, undeclared names, malformed comparisons, and unsupported operators before solver invocation. Integer, real, and Boolean declarations are typed explicitly; variable bounds compile to constraints. The two copies use independent variable maps to prevent accidental aliasing, and model values are normalized before entering a diagnostic record.

Grounding is a deployment gate, not only a prompt instruction. Each repair record contains an action, a formal expression, an exact source span, and an explanation. The gate checks span membership, recompiles the expression, checks $\text{SAT}(F \wedge c)$ and $F \not\models c$, and reruns the target-divergence query. Ablations retain both raw proposals and gated outcomes, so a baseline is not credited under a more permissive acceptance rule. Every result record stores the initial verdict, proposed action, rejection reason, final verdict, model configuration, and prompt hash. The trusted computing base comprises the parser, compiler, solver, and deterministic gate; the language model remains untrusted.

4.4 Repair acceptance semantics

Let $\text{span}(c)$ be the source substring cited for candidate c , and let $\mathcal{R}$ be a frozen set of allowed domain-semantics rules. The gate accepts c exactly when

$$\begin{aligned} \text{Accept}(c) \iff & \text{Compiles}(c) \wedge \text{Grounded}(c, P, \mathcal{R}) \\ & \wedge \text{SAT}(F \wedge c) \wedge \neg(F \models c) \wedge \text{Det}(F \wedge c, q), \quad (2) \end{aligned}$$

where grounding requires either exact occurrence of $\text{span}(c)$ in P or a named rule in $\mathcal{R}$. The policy returns the first accepted candidate within the two-round budget. If neither round yields an accepted addition, it preserves F and returns UNDERSPECIFIED; it never commits a partially validated edit.

This definition prevents three misleading successes. A redundant restatement cannot receive repair credit because $F \models c$. A target pin such as $q = 6$ cannot pass without source support. A grounded but insufficient addition cannot pass if target-divergent models remain. Conversely, the gate deliberately permits a constraint that differs syntactically from c^* when it is source-supported and functionally restores the intended target.

4.5 Solver cost and failure states

The determinacy query duplicates variables and constraints once, so its encoding size is linear in the size of F up to the target disequality. Solver time can still grow nonlinearly with the underlying theory; the implementation therefore preserves timeout and UNKNOWN as first-class states. A determinate input requires no repair-model call. An ambiguous input incurs at most two calls plus solver rechecks, yielding a bounded agent loop rather than unconstrained self-refinement.

The method distinguishes representation failure from mathematical incompleteness. UNSUPPORTED means that the parser or compiler cannot express the attempted formalization. UNKNOWN means that the solver did not decide a supported query. INCONSISTENT means F has no model. Only UNDERSPECIFIED says that the available source did not support an accepted repair for an ambiguous target. Collapsing these states would make both coverage and user-facing behavior uninterpretable.

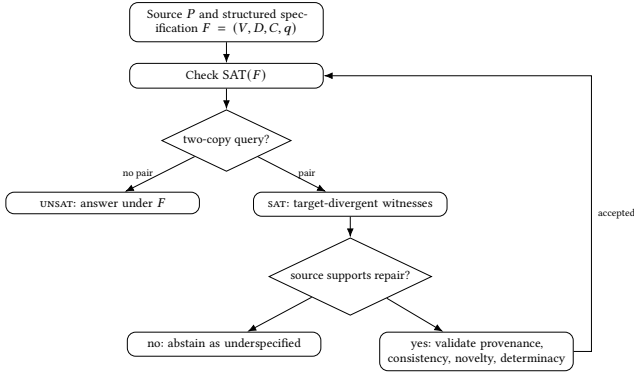


Figure 1: DeGro control flow. Only a solver-validated, source-supported addition re-enters verification; timeouts and unsupported syntax never certify determinacy.

5 Evaluation

5.1 Controlled paired benchmark

For each gold specification $F^* = \{c_1, \dots, c_k, c^*\}$ with target-critical fact c^* , we create $F^- = F^* \setminus \{c^*\}$. We retain an item only when F^* is target-determinate, F^- is satisfiable, and F^- admits target-divergent witnesses. Two cases share exactly F^- : in OMISSION, the source P^+ contains the fact corresponding to c^* ; in UNDERSPECIFIED, P^- omits it. The expected decisions are repair and abstain, respectively. This construction prevents the verifier state alone from revealing the label.

We constructed 120 confirmatory pairs from every remaining compatible MIRA-Math instance after excluding the earlier 80-pair pilot and 100-pair development set [1]. The set contains 240 cases from five families. Selection, source order, prompts, model settings, exclusions, and the dataset hash were frozen before the confirmatory runs. The cumulative analysis combines these 120 pairs with the earlier 180 pairs and is explicitly secondary.

Integrity checks. The builder does not ask an LLM to select or corrupt c^* . It imports the typed missing fact from the MIRA-Math latent state, converts compatible families to MODEL SPEC, and applies the three solver checks before inclusion. A deletion that leaves the target unchanged is therefore never mislabeled as the failure mode of interest. The final subset excludes every earlier pair identifier and exhausts the remaining compatible instances. Its manifest records source revision 72ff918, CC BY 4.0 licensing, selection seed, five family counts, prompt hash, and output hash.

The families exercise distinct constraint structures. Linear-system separation removes a relation distinguishing otherwise compatible solutions. Rank-deficient shared systems retain several degrees of freedom. Graph path sums and Chinese-remainder reconstruction combine discrete equalities with domain restrictions. Moment problems use coupled polynomial summaries within the supported degree bound. Family-stratified analysis checks that an easy template does not dominate the aggregate.

5.2 Methods and hypotheses

The confirmatory comparison includes SELF-REVIEW, GROUNDED SELF-REVIEW, DETERMINACY-GUIDED REPAIR, and DeGro, which combines determinacy feedback with source grounding and solver-validation gates. All four methods reuse the same initial formalization and receive the same two-attempt budget. We ran GPT-OSS 20B and Gemma 4 31B with medium reasoning, temperature 1.0, and a frozen prompt. All $120 \times 2 \times 4 = 960$ expected records were present for each model; no failed calls enter the confirmatory denominator.

An exploratory decision-stage experiment on the disjoint 100-pair development set also compares minimal witnesses, full target witnesses, and DeGro augmented with full witness assignments. These variants were introduced after the initial pilot and are not confirmatory. They address RQ3 by separating the existence of non-uniqueness from the informational content of particular assignments.

The preregistered primary comparison tests whether DeGro improves FDA over grounded self-review. We test it separately for each model with exact paired McNemar tests and Holm correction across the two tests. We report the paired percentage-point difference with a pair-cluster bootstrap 95% interval. Official model documentation describes configurable reasoning effort and structured output support for GPT-OSS [38].

5.3 Metrics and analysis

Primary metrics are repair success rate (RSR), correct abstention rate (CAR), over-repair rate (ORR), and final decision accuracy (FDA). Unsupported repair (UR) is the fraction of proposed additions that fail the source-support criterion. Confidence intervals resample pairs, preserving the omission/underspecification coupling.

RSR requires a grounded, functionally correct addition on omission cases. CAR rewards abstention, rather than an unsupported addition, on underspecified cases. FDA averages the two balanced case types, and ORR is $1 - \text{CAR}$. We retain separate semantic, functional, and grounded repair fields because a constraint can recover the right number by coincidence or cite a relevant sentence while encoding it incorrectly. The deployment score uses the grounded outcome.

5.4 Natural-error study

To test ecological validity, GPT-OSS 20B generated one Structured SymCode specification per MATH-500 problem without injected errors. We included every successfully parsed, solver-supported specification with a conclusive verdict in a cohort frozen before annotation or inspection of repair outcomes. The original problem is the unit of analysis: all methods reuse the same initial specification, preventing shared model generations from being counted as independent observations.

We use dual-model AI annotation with human adjudication and audit. GPT-OSS 20B and Gemma 4 31B independently propose labels and evidence from the source, reference solution, and generated specification under a shared prompt, while blinded to the determinacy verdict, witness, repair proposal, and method outcome. A human resolves every model disagreement and audits all consensus missing-constraint cases plus a frozen, label-stratified sample of the remaining consensus cases. Unreviewed consensus is identified

Table 1: Confirmatory results on 120 preregistered pairs per model. UR is unsupported additions divided by all proposed additions.

Model	Method	RSR↑	CAR↑	FDA↑	ORR↓	UR↓
GPT-OSS 20B	Self-review	93.3	87.5	90.4	12.5	15.2
	Grounded self-review	100.0	80.0	90.0	20.0	16.7
	Determinacy-guided	86.7	99.2	92.9	0.8	6.3
	DeGro	92.5	97.5	95.0	2.5	4.3
Gemma 4 31B	Self-review	100.0	93.3	96.7	6.7	6.3
	Grounded self-review	100.0	95.8	97.9	4.2	4.0
	Determinacy-guided	100.0	99.2	99.6	0.8	0.8
	DeGro	100.0	95.0	97.5	5.0	4.8

as such; we do not describe the corpus as independently human-annotated or fully human-verified. Before resolution we report exact agreement, Cohen’s κ , and agreement on primary-subset membership. The labels are: correct; missing, wrong, or extra constraint; wrong domain; wrong target; computational error; or other. Only a source-supported omission that can change the requested target enters the executable-under-formalization subset.

The primary natural-error estimands are ambiguity-detection recall and end-to-end grounded-repair success within that subset. Secondary measures are repair success conditional on detection, detector precision, false ambiguity and false repair outside the subset, and preservation of adjudicated-correct controls. Proportions receive Wilson 95% intervals. Overall Structured Solver versus DeGro answer accuracy is paired by problem and tested with exact McNemar. A numerically correct answer is not a successful repair unless an accepted grounded addition restores determinacy.

The frozen cohort contains 292 conclusive, solver-supported formalizations from 500 MATH-500 problems; 208 were excluded by deterministic reasons (65 model-level unsupported, 112 solver-level unsupported, 9 inconsistent, 9 unknown, and 13 call errors). Before human resolution, the two annotation models agree on 67.8% of exact labels ($\kappa = .458$) and on 90.8% of primary-subset membership. Ninety-four disagreements and the frozen 44-case consensus audit remain unresolved; consequently, these figures describe annotation reliability and not natural-error prevalence or repair efficacy. We exclude outcome claims rather than treating detector ambiguity as a natural-error label.

6 Confirmatory Results

Table 1 reports the preregistered 120-pair analysis. For GPT-OSS 20B, DeGro raises FDA from 90.0% to 95.0% relative to grounded self-review, a +5.0-point effect (95% CI [0.4, 9.6]; discordant counts 22 versus 10; exact McNemar $p = .050$, Holm $p = .100$). For Gemma 4 31B, FDA changes from 97.9% to 97.5%, a −0.4-point effect (95% CI [−3.3, 2.5]; 5 versus 6; $p = 1.0$). Thus the confirmatory result supports a practically meaningful GPT-OSS gain but not a model-general advantage. The strongest Gemma result is the verdict-only determinacy method (99.6% FDA), showing that the additional grounding interaction can itself introduce errors.

Secondary cumulative analysis. Across all 300 pairs, DeGro versus grounded self-review changes FDA from 91.2% to 96.2% for GPT-OSS (difference +5.0 points, 95% CI [2.3, 7.7]; McNemar $p < .001$)

Table 2: Exploratory GPT-OSS 20B ablation on 100 development pairs.

Method	RSR	CAR	FDA	ORR
Self-review	92	84	88.0	16
Bare non-uniqueness	87	97	92.0	3
Minimal witness	93	93	93.0	7
Target witnesses	86	94	90.0	6
Minimal witness+ground	97	99	98.0	1
DeGro	97	100	98.5	0
DeGro+witnesses	97	98	97.5	2

and from 96.2% to 97.8% for Gemma (difference +1.7 points, 95% CI [−0.2, 3.5]; $p = .099$). These cumulative estimates include development data and do not replace the 120-pair confirmatory tests.

6.1 Witness ablation

Table 2 reports the exploratory 100-pair decision-stage experiment. DeGro and DeGro+witnesses differ by only one FDA point (five versus three discordant decisions favoring DeGro; Holm-adjusted $p = .727$). A minimal witness improves RSR by six points over the bare verdict, but the planned comparison is also non-significant after correction ($p = .714$). Concrete target assignments therefore provide no measurable incremental benefit on this benchmark.

The negative result narrows the contribution. Each controlled instance contains one target-critical omission, so the fact that the target is non-unique may already localize the defect. Witness assignments add tokens but little new information. More complex specifications with nuisance dimensions or several source-supported candidate facts may behave differently, but the present data support determinacy-guided grounding rather than a universal advantage for concrete witnesses.

6.2 Error patterns and cross-model behavior

GPT-OSS errors are asymmetric. On the cumulative set, self-review and grounded self-review repair more than 92% of omissions but abstain correctly on at most 87% of underspecified sources. A bare determinacy verdict reverses the bias, increasing CAR to 98% while reducing RSR to 87.7%. DeGro recovers most lost repairs and preserves 99% CAR. Its unsupported-addition rate among additions is 3.4%, compared with 12.5% for grounded self-review.

Gemma behaves differently: all four cumulative methods repair every omission, so abstention determines the ranking. The bare verdict obtains 99.0% FDA, whereas adding the grounding interaction lowers FDA to 97.8%. The unsupported-addition rates are 2.0% and 4.2%, respectively. The reversal mirrors the confirmatory null result and demonstrates that a stricter instruction is not automatically monotone in safety. Deterministic gating remains necessary even when a prompt explicitly requests evidence.

Family-level results further localize the effect. In the cumulative GPT run, DeGro reaches at least 97.5% FDA in four families but 87.5% in rank-deficient shared systems, where omission repair is hardest. Gemma’s DeGro over-repairs primarily in linear-system separation rather than degrading uniformly. These concentrations argue against summarizing the method with one model-agnostic average.

Table 3: Cumulative FDA by family for grounded self-review (GSR) and DeGro. Each family contains 60 pairs.

Family	GPT-OSS 20B		Gemma 4 31B	
	GSR	DeGro	GSR	DeGro
CRT reconstruction	100.0	99.2	100.0	100.0
Graph path sums	96.7	99.2	99.2	100.0
Linear separation	76.7	97.5	85.8	90.8
Moment problem	88.3	97.5	98.3	100.0
Rank-deficient shared	94.2	87.5	97.5	98.3

Table 4: Frozen MATH-500 cohort accounting for the shared initial structured specification.

Disposition	Count	Percent
Conclusive and solver-supported	292	58.4
Model-level unsupported	65	13.0
Solver-level unsupported	112	22.4
Inconsistent	9	1.8
Solver unknown	9	1.8
Call error	13	2.6
Total	500	100.0

Table 3 shows that the same aggregate effect arises from different mechanisms. GPT’s largest gain is a 20.8-point improvement on linear separation, where generic grounded review often adds an unsupported relation to the incomplete source. Its only negative family is the rank-deficient system, where the determinacy signal encourages abstention even when the complete source supports a repair. Gemma is near ceiling on four families; its smaller aggregate effect is driven by a 5-point improvement on linear separation and correspondingly limited headroom elsewhere.

7 Broad-Benchmark Coverage Audit

The controlled benchmark begins from a validated formalization. To measure the preceding representation bottleneck, GPT-OSS 20B generated one shared Structured SymCode specification for each of the 500 MATH-500 problems [25]. The frozen cohort accounting is complete at the problem level: 292 specifications (58.4%) parsed, reached the solver, and produced a conclusive determinate-or-ambiguous verdict. Table 4 partitions all remaining problems by a deterministic primary exclusion reason.

Coverage, conditional accuracy, and end-to-end accuracy answer different questions. Conditional accuracy measures answer quality among the 292 supported problems; end-to-end accuracy assigns failures to the other 208. Selecting the supported subset after observing answers would inflate performance, which is why the cohort is defined solely from parsing and solver state. The 58.4% coverage also explains why a restricted verifier should not be compared with arbitrary Python generation through one undifferentiated accuracy number.

The exclusion categories also identify engineering priorities. Solver-level unsupported cases are the largest group, indicating that many parsed specifications use constructs outside the chosen theory or compiler. Model-level unsupported cases fail before a

valid MODELSPec exists and therefore require better structured generation rather than a stronger determinacy query. Inconsistent and unknown states are comparatively rare. This decomposition is more actionable than an aggregate “no answer” rate because each category routes to a different component.

The six-method MATH-500 execution logs contain method-specific failed calls beyond the 13 shared-formalizer errors. Consequently, this manuscript does not promote their current accuracy, conditional-accuracy, repair, or cost values to final results. This choice preserves the frozen 500-problem denominator while preventing retry artifacts from being mistaken for method failures. The coverage audit remains valid because it is defined once from the shared initial formalization used to construct the blinded natural-error packet.

8 Validity and Analysis Choices

8.1 Why the paired design matters

A conventional mixture of complete and incomplete questions admits shortcuts: surface wording can reveal missing information, families can differ in difficulty, and a policy can learn that every solver ambiguity should be repaired. Here, the two cases share latent values, target, domains, retained constraints, and solver verdict. Only the sentence corresponding to c^* differs. Pairing therefore removes representation difficulty from the repair-versus-abstain contrast and gives an exact test based on discordant decisions.

The repair policy never receives c^* , the gold target, or the gold action. It receives only the source, current specification, and method-specific diagnostic. Gold latent state is used after generation for scoring. Direct target pins are marked unsupported unless licensed by the source and accepted by the same gate. Raw and gated outcomes reveal when a method appears accurate only because it fabricates a convenient constraint.

8.2 Internal and statistical validity

The primary contrast changes only the determinacy signal: both methods receive the same source, specification, schema, grounding requirement, model, and budget. The final 120 pairs were frozen after excluding all 180 earlier identifiers and before their method outcomes were inspected. The two model-specific tests and their Holm correction were declared in the manifest. No family or case is removed based on a repair result.

The cumulative 300-pair analysis is more precise but includes development data observed earlier. It is therefore labeled secondary throughout. The 100-pair witness ablation is exploratory because its variants were added after the pilot. Keeping these roles separate avoids retroactively treating a favorable cumulative result as the confirmatory test.

8.3 Construct validity

FDA weights omission and underspecification equally because every latent item contributes one of each. This exposes the cost of over-repair, but deployments may assign a larger penalty to fabricated facts than to missed repairs. The released case-level records permit alternative utilities. UR is conditioned on proposing an addition and must be read with RSR and CAR: a method that always abstains has zero unsupported additions and zero repair utility.

Exact-span provenance is an auditable proxy, not semantic entailment between language and logic. A model can cite the right sentence and encode the wrong equation. Conversely, implicit mathematical conventions may justify an expression without a contiguous span. Separate semantic, functional, grounding, and determinacy fields expose these disagreements instead of collapsing them into numeric answer accuracy.

8.4 External validity

MIRA-derived cases are templated and remove exactly one known fact. They provide causal control but underrepresent paraphrase, irrelevant prose, diagrams, implicit conventions, wrong constraints, and interacting translation errors. MATH-500 offers more natural language but exposes the opposite limitation: much of it lies outside the present arithmetic SMT fragment. The evaluations answer different questions and are not merged into a leaderboard score.

Transfer is also bounded. GPT-OSS 20B and Gemma 4 31B differ in post-training and instruction behavior; their divergent results are direct evidence against a universal prompt claim. Hosted checkpoints can change, so the artifact records exact model strings, timestamps, reasoning settings, temperature, prompt hashes, and raw responses.

8.5 Natural-error validity

The frozen natural-error protocol avoids selecting examples from detector outputs, but its current labels still depend partly on two AI annotators. Exact agreement of 67.8% indicates that the detailed taxonomy is nontrivial, while 90.8% agreement on primary-subset membership suggests greater stability for the paper’s narrower estimand. Human review of all 94 disagreements and 44 selected consensus cases is mandatory under the frozen protocol; until those fields are resolved, prevalence and repair rates are not statistically identified.

This distinction prevents circular validation. DeGro’s ambiguous verdict is hidden from annotators and cannot define the positive class. A positive case must instead contain a source-supported target-critical omission established from the source and generated specification. Correct, wrong-constraint, wrong-target, and wrong-domain cases remain in the denominator used to measure false ambiguity, false repair, and preservation.

9 Agent Integration

DeGro is a verifier policy inside a larger agent, not a standalone solver. A host agent first parses the task and produces an executable artifact. The verifier then returns a typed observation that a controller maps to an action, as summarized in Table 5. Keeping the states distinct matters: UNKNOWN denotes solver inconclusiveness, UNSUPPORTED denotes a representation limitation, and UNDERSPECIFIED is a conclusion about missing source support.

In an interactive setting, UNDERSPECIFIED can trigger a clarification request rather than terminate the task. For the running example, the agent can explain that (4, 6) and (6, 4) both satisfy the stated total and ask which additional relationship holds. The controlled benchmark scores abstention because no information channel is available; it does not claim that stopping is preferable to clarification.

Table 5: Controller semantics for DeGro integration.

Verifier state	Evidence	Controller action
Determinate	F sat, Φ unsat	answer with scope
Ambiguous	targets disagree	grounded repair
Inconsistent	F unsat	revise encoding
Unknown	solver inconclusive	fallback/escalate
Unsupported	outside fragment	alternate tool
Underspecified	no grounded repair	ask or abstain

The verifier and repair model can also be separated. One component generates F , a deterministic tool checks determinacy, and another component proposes a grounded action. This reduces self-confirmation and produces an audit trail linking each accepted repair to a source span and two solver checks. Interfaces should phrase a successful result as “the encoded constraints determine the target,” not “the problem has been formally proved.”

10 Discussion

What the evidence supports. The paired construction demonstrates that a solver state can be held constant while the correct action changes with the source. For GPT-OSS 20B, determinacy-guided grounding improves the joint decision and reduces unsupported additions. The mechanism is most useful as a guard against generic review that fills a formal gap with a plausible but unstated relation.

What the evidence does not support. The study does not establish semantic equivalence between P and F , a universal benefit from grounding prompts, or an incremental benefit from concrete witnesses. It also does not show superiority over unrestricted program generation on MATH-500. The Gemma result and the incomplete broad-benchmark method logs make these boundaries empirical findings rather than boilerplate caveats.

Why the negative witness result matters. The two-copy query remains essential for computing the determinacy verdict and can return an auditable counterexample. Nevertheless, exposing the full pair to the repair model does not improve decisions on the present data. A simpler deployed policy can therefore communicate only the typed verdict by default and retain witnesses for audit, explanation, or harder cases. This saves context while preserving the formal check.

Relation to selective action. Confidence-based abstention asks whether a predictor appears uncertain. DeGro asks a different question: whether the encoded constraints logically permit more than one target and whether the source licenses a restriction. The two signals can be composed. A determinate target with low language-model confidence may still deserve fallback, while an ambiguous target should not be authorized merely because the model is confident.

11 Artifact and Reproducibility

The artifact package contains the typed compiler and two-copy solver query, repair gates, unit tests, dataset builders, frozen manifests, prompts and schemas, raw JSONL records, and analysis scripts.

The confirmatory manifest records the MIRA-Math source revision and license, selection seed, five family counts, exact model strings, decoding settings, prompt SHA-256, and output SHA-256. Its 120-pair output hash is 5616d0c6. . . 4ce3f0b; each model analysis consumes all 960 expected records.

Analysis scripts reconstruct tables from JSONL and fail when expected pair counts are missing. The cumulative assembler records whether a row originated in the pilot, development, or confirmatory subset. This permits independent reproduction of the 120-pair headline without incorporating earlier observations. Statistical code implements pair-cluster bootstrap intervals, exact two-sided McNemar tests, and Holm adjustment.

The natural-error packet is separately blinded and content-addressed. Its 292-record packet hash is a7f16240. . . 52e063; the frozen human-review queue contains 94 disagreements and 44 consensus audits with hash ba43d260. . . 6d6009. Detector verdicts, repairs, method names, and answer outcomes are absent from annotation inputs. No credential, provider account identifier, or secret is included in the release.

12 Limitations and Risks

Target determinacy is neither semantic faithfulness nor answer correctness relative to the source. Wrong or extra constraints can force a unique incorrect target and evade the check. An omission that does not affect q is likewise invisible. The current SMT fragment excludes much nonlinear, transcendental, and diagrammatic mathematics, so accuracy without coverage is misleading. Witnesses may differ on irrelevant variables, multiple omissions can exceed the bounded repair budget, and exact-span grounding underhandles implicit mathematical knowledge.

The controlled study removes one known target-critical constraint and therefore estimates a repair decision under idealized formalization control. The unresolved human review prevents the frozen natural-error cohort from supporting prevalence, detection, repair, or preservation claims. The existing MATH-500 method logs also contain failed calls, so no end-to-end MATH-500 accuracy or cost comparison is reported. These omissions are explicit evidence boundaries, not zero-valued outcomes.

The method can increase false confidence if representation-relative determinacy is presented as truth. Interfaces should expose the constraints and any cited repair rather than display an unqualified certification. The evaluated datasets contain generated or public mathematical problems and no personal data. AI assistance used in study design, implementation, experimentation, or writing requires venue-compliant disclosure; authors remain responsible for every claim, citation, and released artifact.

13 Conclusion

Execution answers whether an artifact runs; DeGro asks whether its feasible models agree on the quantity the agent is about to report. Grounded repair-or-abstain then separates a recoverable translation omission from genuinely missing information. On the preregistered 120-pair set, DeGro improves GPT-OSS 20B by five FDA points but does not improve Gemma 4 31B, ruling out a model-independent gain. Concrete witnesses provide no measurable advantage over the verdict-plus-grounding policy. The resulting claim is narrow but

actionable: target determinacy is a checkable intermediate safety condition, and its utility must be reported together with grounding behavior, representation coverage, and the semantic boundary between source and formalization.

References

- [1] Charbel Al Bateh and Samer Saab Jr. 2026. MIRA-Math: A Benchmark for Minimal Information Requesting and Mathematical Reasoning. *arXiv preprint arXiv:2607.07391* (2026).
- [2] Mohammadh Alhanahnah, Md Rashedul Hasan, and Hamid Bagheri. 2024. An Empirical Evaluation of Pre-trained Large Language Models for Repairing Declarative Formal Specifications. *arXiv preprint arXiv:2404.11050* (2024).
- [3] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *International Conference on Learning Representations*.
- [4] Paul C. Attie, Kinan Dak Al Bab, and Mouhammad Sakr. 2017. Model and Program Repair via SAT Solving. *ACM Transactions on Embedded Computing Systems* 17, 2, Article 32 (2017). doi:10.1145/3147426
- [5] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. 2023. ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. *arXiv preprint arXiv:2302.12433* (2023).
- [6] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Qiaocho Jiang, Jia Deng, Stella Biderman, and Sean Welleck. 2024. Llemma: An Open Language Model for Mathematics. In *International Conference on Learning Representations*.
- [7] Gilles Barthe, Juan Manuel Crespo, and César Kunz. 2011. Relational Verification Using Product Programs. In *FM 2011: Formal Methods (Lecture Notes in Computer Science, Vol. 6664)*. 200–214. doi:10.1007/978-3-642-21437-0_17
- [8] Gilles Barthe, Pedro R. D’Argenio, and Tamara Rezk. 2004. Secure Information Flow by Self-Composition. In *17th IEEE Computer Security Foundations Workshop*. 100–114. doi:10.1109/CSFW.2004.1310735
- [9] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023).
- [10] Edmund M. Clarke, Orna Grumberg, Suresh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 1855)*. 154–169. doi:10.1007/10722167_15
- [11] Michael R. Clarkson, Bernd Finkbeiner, Masoud Koeini, Kristopher K. Micinski, Markus N. Rabe, and César Sánchez. 2014. Temporal Logics for Hyperproperties. In *Principles of Security and Trust (Lecture Notes in Computer Science, Vol. 8414)*. 265–284. doi:10.1007/978-3-642-54792-8_15
- [12] Michael R. Clarkson and Fred B. Schneider. 2010. Hyperproperties. *Journal of Computer Security* 18, 6 (2010), 1157–1210. doi:10.3233/JCS-2009-0393
- [13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. In *arXiv preprint arXiv:2110.14168*.
- [14] Corinna Cortes, Giulia DeSalvo, and Mehryar Mohri. 2016. Learning with Rejection. In *Algorithmic Learning Theory (Lecture Notes in Computer Science, Vol. 9925)*. 67–82. doi:10.1007/978-3-319-46379-7_5
- [15] Wenyao Cui, Huaping Zhang, Yongyi Huang, Qiuchi Li, Jian Xu, Cheng-Lin Liu, Chunxiao Gao, Juan Wang, and Baohua Zhang. 2026. SymDiag: Explainable Diagnosis for LLM Reasoning via Neuro-Symbolic Verification. *arXiv preprint arXiv:2608.08786* (2026).
- [16] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [17] Isil Dillig, Thomas Dillig, Boyang Li, and Kenneth L. McMillan. 2013. Inductive Invariant Generation via Abductive Inference. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 443–456. doi:10.1145/2491956.2462187
- [18] Ran El-Yaniv and Yair Wiener. 2010. On the Foundations of Noise-Free Selective Classification. *Journal of Machine Learning Research* 11 (2010), 1605–1641.
- [19] Bernd Finkbeiner, Markus N. Rabe, and César Sánchez. 2015. Model Checking HyperLTL. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 9206)*. 30–48. doi:10.1007/978-3-319-21690-4_3
- [20] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *International Conference on Machine Learning*.
- [21] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [22] Yonatan Geifman and Ran El-Yaniv. 2019. SelectiveNet: A Deep Neural Network with an Integrated Reject Option. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. 2151–2159.
- [23] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.
- [24] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In *International Conference on Learning Representations*.
- [25] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1.
- [26] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*.
- [27] Albert Qiaocho Jiang, Wenda Li, and Mateja Jamnik. 2024. Multilingual Mathematical Autoformalization. In *International Conference on Learning Representations*.
- [28] Albert Qiaocho Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2023. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. In *International Conference on Learning Representations*.
- [29] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislaw Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, and Jared Kaplan. 2022. Language Models (Mostly) Know What They Know. *arXiv preprint arXiv:2207.05221* (2022).
- [30] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations*.
- [31] Junbo Jacob Lian, Yujun Sun, Huiling Chen, Chaoyu Zhang, Hanzhang Qin, and Chung-Piaw Teo. 2026. ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization. *arXiv preprint arXiv:2602.15983* (2026).
- [32] Mark H. Liffiton and Karem A. Sakallah. 2008. Algorithms for Computing Minimal Unsatisfiable Subsets of Constraints. *Journal of Automated Reasoning* 40 (2008), 1–33. doi:10.1007/s10817-007-9084-z
- [33] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023).
- [34] Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful Chain-of-Thought Reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. 305–329. doi:10.18653/v1/2023.ijcnlp-main.20
- [35] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [36] Hussein Mozannar and David Sontag. 2020. Consistent Estimators for Learning to Defer to an Expert. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. 7076–7087.
- [37] Sina Bagheri Nezhad, Yao Li, and Ameeta Agrawal. 2026. SymCode: A Neurosymbolic Approach to Mathematical Reasoning via Verifiable Code Generation. In *Findings of the European Chapter of the Association for Computational Linguistics*. https://arxiv.org/abs/2510.25975
- [38] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. Accessed 2026-09-09. https://openai.com/index/gpt-oss-model-card/
- [39] Raymond Reiter. 1987. A Theory of Diagnosis from First Principles. *Artificial Intelligence* 32, 1 (1987), 57–95. doi:10.1016/0004-3702(87)90062-2
- [40] Dipankar Sarkar. 2026. From Errors to Proofs: Minimal-Core-Guided Repair for Neuro-Symbolic Constraint Solving. *arXiv preprint arXiv:2608.14771* (2026).
- [41] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [42] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*. 404–415. doi:10.1145/1168857.1168907
- [43] Marcelo Sousa and Isil Dillig. 2016. Cartesian Hoare Logic for Verifying k-Safety Properties. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 57–69. doi:10.1145/2908080.2908092
- [44] Tachio Terauchi and Alex Aiken. 2005. Secure Information Flow as a Safety Problem. In *Static Analysis (Lecture Notes in Computer Science, Vol. 3672)*. 352–367.
- [45] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving Math Word Problems with Process- and Outcome-Based Feedback. *arXiv preprint arXiv:2211.14275* (2022).
- [46] Shyam Upadhyay and Ming-Wei Chang. 2017. Annotating Derivations: A New Evaluation Strategy and Dataset for Algebra Word Problems. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational*

- Linguistics: Volume 1, Long Papers*. Association for Computational Linguistics, 494–504. <https://aclanthology.org/E17-1047/>
- [47] Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2024. MathCoder: Seamless Code Integration in LLMs for Enhanced Mathematical Reasoning. In *International Conference on Learning Representations*.
 - [48] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.
 - [49] Yixuan Weng, Minjun Zhu, Shizhu He, Kang Liu, and Jun Zhao. 2023. Large Language Models Are Better Reasoners with Self-Verification. *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023).
 - [50] Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35.
 - [51] Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. 2024. DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data. *arXiv preprint arXiv:2405.14333* (2024).
 - [52] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36.
 - [53] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
 - [54] Huaiyuan Ying, Zijian Wu, Yihan Geng, Jiayu Wang, Dahua Lin, and Kai Chen. 2024. Lean Workbook: A Large-Scale Lean Problem Set Formalized from Natural Language Math Problems. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.
 - [55] Zhouliang Yu, Weijie Chen, Jiayu Chen, Hongming Zhou, Kaifeng He, et al. 2025. FormalMATH: Benchmarking Formal Mathematical Reasoning of Large Language Models. *arXiv preprint arXiv:2505.02735* (2025).
 - [56] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200. doi:10.1109/32.988498
 - [57] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. 2022. miniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics. In *International Conference on Learning Representations*.